

ANEXO TÉCNICO

Manual de despliegue en AWS y GCP con Docker y GitHub

Guía paso a paso para llevar el agente cotizador de equipos médicos a producción sin costo, usando control de versiones, contenedores y dos proveedores de nube.

Webinar: Desarrollo y Puesta en Producción de Agentes de Inteligencia Artificial en la Nube

Luis David Chuquín Itas

Instituto Superior Tecnológico Cenestur · Quito, Ecuador · 2026

Índice

1. Introducción y alcance de este manual
2. Crear tus cuentas en la nube — AWS y GCP, y qué créditos dan a cuentas nuevas
3. Docker — empaquetar cualquier servicio en un contenedor
4. GitHub — control de versiones y base del despliegue continuo
5. Opción A: desplegar en AWS — CloudFront o la ruta completa con ECS Fargate
6. Opción B: desplegar en GCP — Cloud Run
7. Opción C: desplegar gratis con GitHub + Render — sin proveedor de nube
8. Comparación de costos — tabla resumen
9. Combinando piezas — usar CloudFront como entrada de cualquier opción
10. Referencias

1. Introducción y alcance de este manual

Este documento es un anexo técnico complementario a la propuesta del webinar "*Desarrollo y Puesta en Producción de Agentes de Inteligencia Artificial en la Nube*". Su objetivo es presentar, de forma reproducible, las opciones disponibles para llevar un agente como el cotizador de equipos médicos desde el código fuente hasta una URL pública funcionando: una ruta en **AWS**, una ruta en **GCP**, y una ruta completamente **gratuita** basada en **GitHub** y **Render** que no depende de ningún proveedor de nube tradicional. Las tres usan **Docker** como formato común de empaquetado, lo que permite moverse de una opción a otra sin cambiar el código de la aplicación.

No es un manual genérico de "qué es la nube": asume que quien lo lee ya vio la explicación conceptual del webinar, y aquí se profundiza en el cómo, con comandos reales para cada opción y las consideraciones de costo y facilidad que hacen preferible una u otra según el caso.

2. Crear tus cuentas en la nube

2.1 Cuenta de AWS

- 1 Entra a `https://aws.amazon.com/` y haz clic en "Crear una cuenta de AWS".
- 2 Regístrate con un correo electrónico y define el nombre de la cuenta (esta será tu cuenta **root**, con acceso total — úsala solo para tareas administrativas, no para el día a día).
- 3 AWS pide una tarjeta de crédito/débito para verificar identidad, incluso en el plan gratuito. Realiza un cargo pequeño temporal (usualmente reversado) para confirmarla.
- 4 Verifica tu identidad por teléfono (llamada o SMS con un código).
- 5 Elige un plan de soporte — "Basic support - Free" es suficiente para empezar.
- 6 Crea un usuario IAM aparte para trabajar (nunca uses las credenciales root en el día a día) con permisos acotados a lo que vayas a usar (ver sección 5).

💰 Créditos para cuentas nuevas de AWS

Desde julio de 2025, las cuentas nuevas reciben **USD 100** de crédito al registrarse, y pueden llegar a **USD 200** completando actividades de incorporación (por ejemplo, usar EC2 o Bedrock). El plan gratuito dura hasta **6 meses o hasta que se agote el crédito**, lo que ocurra primero. Además de esto, más de 30 servicios tienen una capa **"Always Free"** que no depende de esos créditos ni de la antigüedad de la cuenta — por ejemplo, CloudFront (1 TB de salida + 10 millones de solicitudes al mes, para siempre). Este proyecto se apoya en esa capa permanente, no en los créditos de bienvenida.

2.2 Cuenta de GCP (Google Cloud)

- 1 Entra a `https://console.cloud.google.com/` con una cuenta de Google (Gmail u otra) — de preferencia una **personal**, distinta de tu correo de trabajo si aplica, para no mezclar proyectos.
- 2 Acepta activar la "Prueba gratuita" cuando el asistente te lo proponga.
- 3 Se te pedirá una tarjeta para verificación — GCP aclara explícitamente que **no cobra automáticamente** al terminar la prueba; hay que activar manualmente un plan de pago para seguir usando el servicio después.
- 4 Crea un proyecto nuevo (`gcloud projects create <id>` o desde la consola) para aislar este trabajo de cualquier otro proyecto que ya tengas.
- 5 Vincula una cuenta de facturación al proyecto — es un requisito técnico de casi todos los servicios de cómputo (incluido Cloud Run), incluso si te vas a quedar dentro de la capa gratuita.

💰 Créditos para cuentas nuevas de GCP

Las cuentas nuevas de Google Cloud reciben **USD 300** en créditos, válidos por **90 días**. Aparte de esa prueba, GCP también mantiene un nivel **"Always Free"** permanente en varios servicios — el relevante para este proyecto es **Cloud Run**: 2 millones de solicitudes, 180.000 vCPU-segundos y 360.000 GiB-segundos de memoria al mes, para siempre, independientemente de los créditos de bienvenida.

⚠️ Con qué te puedes tropezar

Si la cuenta de facturación de un proyecto de GCP queda **cerrada** (por ejemplo, tras cancelar una tarjeta), habilitar cualquier servicio de cómputo —incluido Cloud Run— falla con `Billing account ... is not found` hasta reactivarla o vincular una nueva desde <https://console.cloud.google.com/billing> . En AWS, de forma similar, una cuenta puede quedar con un bloqueo de "*debe verificarse*" para crear ciertos recursos nuevos (por ejemplo, distribuciones de CloudFront) — se resuelve generalmente desde la propia consola o abriendo un caso en AWS Support; en ambos casos es un tema de verificación de cuenta, no un problema de permisos ni de código.

3. Docker: empaquetar el agente en un contenedor

¿Qué es **Docker**? Una herramienta que empaqueta una aplicación junto con todo lo que necesita para correr (versión de Python, librerías, configuración del sistema) en una unidad llamada **imagen**. Un contenedor es una instancia en ejecución de esa imagen. La ventaja central: si la imagen corre en tu laptop, corre exactamente igual en cualquier servidor, nube o plataforma que sepa ejecutar contenedores — se elimina el clásico "en mi máquina sí funciona".

¿Por qué es **gratis**? Docker Engine (el motor que construye y corre contenedores) es de código abierto y gratuito. Docker Desktop (la app con interfaz gráfica) tiene un plan free para uso personal/educativo. No se paga nada por usar contenedores en sí — lo que cuesta, si acaso, es el cómputo donde terminan corriendo (un servidor, un servicio de nube).

El Dockerfile de este proyecto, explicado

```
FROM python:3.11-slim          # imagen base: Python 3.11, ligera

ENV PYTHONDONTWRITEBYTECODE=1  # no genera archivos .pyc
ENV PYTHONUNBUFFERED=1         # logs salen al instante (no en buffer)

WORKDIR /app                   # carpeta de trabajo dentro del contenedor

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt  # instala dependencias

COPY . .                       # copia el resto del código

RUN mkdir -p /app/data/quotes  # carpeta para los PDF generados

EXPOSE 8080                    # documenta el puerto que usa el servicio
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```

El `${PORT:-8080}` del último paso es lo que permite que la misma imagen funcione tanto en local (puerto 8080 fijo) como en plataformas que asignan su propio puerto dinámicamente, como Render.

Construir y probar la imagen en local

```
docker build -t agente-cotizador-webinar .
docker run -p 8080:8080 agente-cotizador-webinar
curl http://localhost:8080/health
```

4. GitHub: control de versiones y base del despliegue

¿Qué es y por qué importa aquí? GitHub aloja el código fuente con control de versiones (Git): cada cambio queda registrado, es reversible, y varias personas pueden colaborar sin pisarse el trabajo. Pero en un despliegue moderno, GitHub cumple un segundo rol igual de importante: es la **fuentes de verdad** desde la que las plataformas de hosting (Render, y en general cualquier proveedor con integración continua) construyen y publican la aplicación automáticamente.

Cómo funciona el flujo típico

- 1 El código (con su `Dockerfile`) vive en un repositorio de GitHub.
- 2 Cada `git push` a la rama principal notifica a la plataforma de hosting mediante un webhook.
- 3 La plataforma (Render, Cloud Run vía Cloud Build, etc.) clona el repositorio, detecta el `Dockerfile` y construye la imagen automáticamente — sin subir manualmente ningún artefacto.
- 4 Si la construcción y el healthcheck (`/health`) pasan, la plataforma reemplaza la versión anterior sin caída de servicio perceptible.

Este patrón —código en git, build automático en cada push— es la base de lo que en la industria se llama **CI/CD** (integración y despliegue continuos). No hace falta un servidor propio de CI: la mayoría de plataformas gratuitas (Render, GitHub Pages, Vercel, Cloud Run con Cloud Build) ya lo incluyen al conectar el repositorio.

Buenas prácticas recomendadas

- `.gitignore` excluye `.env`, la base de datos SQLite local y los PDF generados — nada de eso debe viajar al repositorio.
- Los secretos reales (claves de API, contraseñas de Redis) solo existen como variables de entorno en la plataforma de hosting, nunca como texto en el código.
- Repositorios públicos en GitHub son gratis sin límite; los privados también lo son para cuentas personales, con colaboradores ilimitados.

5. Opción A: desplegar en AWS

AWS ofrece (al menos) dos rutas razonables para este tipo de servicio, según cuánto control y presupuesto se tenga disponible.

5.1 Ruta rápida y gratuita: CloudFront como entrada pública

CloudFront es la red de distribución de contenido (CDN) de AWS: entrega contenido desde ubicaciones ("edge locations") cercanas a cada usuario, y también puede actuar como un proxy HTTPS hacia cualquier origen — incluido un backend desplegado fuera de AWS (ver Opción C, sección 7). Es la forma más rápida de tener una URL de AWS pública sin costo.

```
# Crear la distribución (el origen puede ser cualquier backend HTTPS público)
aws cloudfront create-distribution \
  --distribution-config file://cf-distribution-config.json \
  --profile <tu-perfil>

# Verificar que terminó de propagarse (puede tardar 5-15 min)
aws cloudfront get-distribution --id <ID> \
  --query 'Distribution.Status' --profile <tu-perfil>
```

⚠ Detalle importante si el origen es una plataforma multi-tenant (Render, Vercel...)

La política de origen request debe ser **"Managed-AllViewerExceptHostHeader"**, no "AllViewer". Con "AllViewer", CloudFront reenvía el header `Host` del visitante (el dominio de CloudFront) en vez del dominio real del origen, y plataformas que enrutan por ese header responden 502. Con "AllViewerExceptHostHeader", CloudFront corrige el `Host` automáticamente al del origen.

5.2 Ruta completa 100% AWS: ECR + ECS Fargate + ALB

Para no depender de ningún servicio externo, la aplicación puede correr enteramente dentro de AWS — la opción con más control, pensada para tráfico real sostenido:

- 1 **ECR** (Elastic Container Registry): registro privado donde se publica la imagen Docker construida en la sección 3.
- 2 **ECS Fargate**: ejecuta el contenedor sin administrar servidores — se define cuánta CPU/memoria necesita y AWS se encarga del resto.
- 3 **Application Load Balancer (ALB)**: reparte el tráfico entrante hacia las instancias del contenedor y expone un DNS público estable.
- 4 **Secrets Manager**: guarda credenciales (API keys, contraseñas) sin exponerlas en variables de entorno planas ni en el código.
- 5 **CloudWatch**: recibe los logs automáticamente y permite configurar alarmas (por ejemplo, si `/health` deja de responder).

🚩 Esta ruta no es gratis

A diferencia de CloudFront, el ALB tiene un costo fijo (~USD 16-20/mes) incluso sin tráfico, y Fargate cobra por tiempo de cómputo reservado. Conviene solo cuando el tráfico esperado justifica ese gasto fijo — para una demo puntual, la ruta 5.1 (o la Opción C de la sección 7) es más razonable. El comando de referencia completo está en `infra/DEPLOY_AWS.md` del repositorio.

6. Opción B: desplegar en GCP con Cloud Run

Cloud Run es el servicio "serverless" de contenedores de Google Cloud: recibe una imagen Docker, la ejecuta bajo demanda, y **escala a cero** cuando nadie la está usando — es decir, no se paga nada mientras no hay tráfico. Es una alternativa directa a la Opción A cuando se prefiere el ecosistema de Google o ya se tiene experiencia con él.

```
# 1. Autenticarse con la cuenta que va a usarse para este proyecto
gcloud auth login

# 2. Crear un proyecto dedicado (evita mezclar con otros proyectos personales/de trabajo)
gcloud projects create <id-del-proyecto> --name="<Nombre del proyecto>"
gcloud config set project <id-del-proyecto>

# 3. Habilitar el servicio (requiere facturación vinculada, ver sección 2.2)
gcloud services enable run.googleapis.com

# 4. Desplegar directo desde el código fuente (Cloud Build construye la imagen)
gcloud run deploy <nombre-del-servicio> \
  --source . \
  --region us-east1 \
  --allow-unauthenticated \
  --port 8080
```

Nota práctica

El paso 3 requiere una cuenta de facturación **activa** vinculada al proyecto. Si falla con `Billing account for project ... is not found`, la causa habitual es una cuenta de facturación cerrada (por ejemplo, tras cancelar la tarjeta asociada) — se resuelve reactivándola o vinculando una nueva desde <https://console.cloud.google.com/billing> antes de reintentar.

7. Opción C: gratis con GitHub + Render (sin proveedor de nube)

Cuando no se necesita infraestructura propia — el caso típico de una demo, un proyecto académico o un prototipo — la combinación más simple es dejar que una plataforma de hosting gratuita construya y sirva la misma imagen Docker directamente desde GitHub, sin pasar por AWS ni GCP en absoluto.

- 1 Subir el proyecto (con su `Dockerfile`) a un repositorio de GitHub.
- 2 Crear una cuenta gratuita en render.com — no pide tarjeta para el plan free.
- 3 "New Web Service" → conectar el repositorio → Render detecta el `Dockerfile` y construye la imagen automáticamente.
- 4 Cada `git push` siguiente dispara un nuevo build y despliegue solo.

✓ **Cuándo conviene esta opción**

Es la ruta más rápida de poner en marcha y la única de las tres sin ningún requisito de tarjeta o verificación de cuenta. La limitación a considerar: el plan free de Render "duerme" el servicio tras ~15 minutos sin tráfico (la siguiente solicitud tarda hasta ~1 minuto en responder mientras despierta) y da 750 horas de cómputo al mes.

8. Comparación de costos

Herramienta / servicio	¿Cuesta algo?	Detalle
GitHub	Gratis	Repos públicos y privados ilimitados, colaboradores ilimitados.
Docker	Gratis	Motor y CLI de código abierto; Docker Desktop free para uso personal.
Render (Opción C)	Gratis	750h/mes, sin tarjeta; se "duerme" tras ~15 min sin tráfico.
AWS CloudFront (Opción A, ruta 5.1)	Gratis (permanente)	1 TB de salida + 10M solicitudes/mes, para siempre, cualquier cuenta.
AWS ECS Fargate + ALB (Opción A, ruta 5.2)	De pago	~USD 25-35/mes corriendo continuamente.
GCP Cloud Run (Opción B)	Gratis (permanente), requiere tarjeta verificada	2M solicitudes + 180.000 vCPU-s/mes, para siempre.
Redis Cloud (checkpointer, opcional en cualquier opción)	Gratis	Plan free de Redis Cloud, suficiente para una demo.
Cloudflare Tunnel (respaldo para pruebas locales)	Gratis	Sin cuenta; URL temporal, no apto para dejarlo corriendo días.

9. Combinando piezas: CloudFront como entrada de cualquier opción

Las tres opciones no son excluyentes. Como CloudFront (sección 5.1) acepta cualquier origen HTTPS público, puede colocarse delante de un backend en ECS Fargate (Opción A completa), en Cloud Run (Opción B) o en Render (Opción C) indistintamente — por ejemplo, para tener una URL de AWS aunque el cómputo real corra gratis en otra plataforma:

GitHub (código + build automático) → **Docker** (imagen construida desde el `Dockerfile`) → backend en **Render**, **Cloud Run** o **ECS Fargate** (cualquiera de las tres opciones) → opcionalmente **AWS CloudFront** como entrada pública (free tier permanente) → usuario final.

Elegir entre estas combinaciones es, en el fondo, una decisión de costo y control: la Opción C es la más económica y rápida de configurar; la Opción A completa (5.2) da más control operativo dentro de AWS a cambio de un costo fijo mensual; la Opción B es equivalente a la Opción C en costo pero dentro del ecosistema de Google.

10. Referencias

- AWS Free Tier — <https://aws.amazon.com/free/>
- Google Cloud Free Program — <https://cloud.google.com/free>
- Documentación de Docker — <https://docs.docker.com/>
- Documentación de GitHub — <https://docs.github.com/>
- Render — <https://render.com/docs>
- Repositorio del proyecto — <https://github.com/mathsci-2510/agente-cotizador-webinar>